

Python para Backend

Django Rest Framework

Javier Villegas

12 de mayo 2025



Construyendo APIs con Django REST Framework

Este laboratorio te guiará a través de la creación de una API RESTful completa utilizando Django REST Framework (DRF). Trabajaremos con el sistema POS existente para crear endpoints que permitan acceder y manipular datos de artículos, órdenes y usuarios.

Vamos a implementar:

1. Serializers (incluyendo serializers dinámicos y anidados)
2. Vistas basadas en funciones y clases
3. Mixins y vistas genéricas
4. ViewSets y enrutamiento
5. Filtrado y paginación
6. Throttling
7. Autenticación y seguridad
8. Validaciones
9. Versionamiento de APIs
10. Manejo de errores y personalización de respuestas

Introducción a Django Rest Framework

¿Qué es REST?

REST (Representational State Transfer) es un estilo arquitectónico para diseñar aplicaciones en red. Se basa en la comunicación cliente-servidor sin estado, normalmente a través de HTTP.

Principios clave:

- Sin estado: Cada solicitud contiene toda la información necesaria para procesarla.
- Basado en recursos: Las API se organizan en torno a recursos (entidades como artículos, productos, usuarios)
- Utiliza métodos HTTP estándar: GET, POST, PUT, DELETE, etc.
- Orientado a la representación: Los recursos pueden tener múltiples representaciones (JSON, XML, etc.)

¿Qué es Django REST Framework?

Django REST Framework es un potente conjunto de herramientas para construir APIs Web en Django. Proporciona:

- Serialización para convertir modelos Django a JSON/XML
- Autenticación y permisos
- Interfaz API navegable
- Viewsets y routers para un desarrollo rápido
- Amplia documentación y soporte de la comunidad

Ahora vamos a instalar Django Rest Framework, para ello ejecutamos lo siguiente

Unset

```
pip install djangorestframework
pip install markdown          # Para navegación por la API
pip install django-filter    # Para filtrado
```

Ahora vamos a registrar el paquete en nuestro pos_project/[settings.py](#)

Python

```
INSTALLED_APPS = [  
    # existing apps  
    'rest_framework',  
]
```

Ahora vamos a agregar una nueva aplicación donde implementaremos nuestra API, para ello ejecutamos lo siguiente:

Unset

```
django-admin startapp api
```

Configura las rutas en urls.py

Para configurar las rutas, vamos a agregar el archivo `post_project/urls.py` y configurar lo siguiente:

Python

```
# pos_project/urls.py  
  
from django.urls import path, include  
  
urlpatterns = [  
    # ...  
    path('api/', include('api.urls')),  
    # ...  
]
```

Serializadores

Los serializadores convierten datos complejos (como instancias de modelos Django) en tipos de datos nativos de Python que pueden ser renderizados en JSON, XML u otros tipos de contenido. También deserializan los datos analizados a tipos complejos.

Serializador básico

Para crear un serializador básico se necesita importar la clase `serializers` de `rest_framework` y definir campos para un serializador igual que se crea un formulario o modelo en Django. Vamos a crear nuestro primer serializador para ello vamos a crear el archivo `api/serializers.py`, y escribimos lo siguiente:

Python

```
# api/serializers.py

from rest_framework import serializers

from core.models import Artículo, ListaPrecio, GrupoArticulo, LineaArticulo


class ArtículoSerializer(serializers.Serializer):

    articulo_id = serializers.UUIDField(read_only=True)

    codigo_articulo = serializers.CharField(max_length=25)

    codigo_barras = serializers.CharField(max_length=25, required=False,
allow_blank=True)

    descripcion = serializers.CharField(max_length=150)

    presentacion = serializers.CharField(max_length=100, required=False,
allow_blank=True)

    stock = serializers.DecimalField(max_digits=12, decimal_places=2,
default=0)

    def create(self, validated_data):
```

```

        return Artículo.objects.create(**validated_data)

    def update(self, instance, validated_data):
        instance.codigo_articulo = validated_data.get('codigo_articulo',
instance.codigo_articulo)

        instance.codigo_barras = validated_data.get('codigo_barras',
instance.codigo_barras)

        instance.descripcion = validated_data.get('descripcion',
instance.descripcion)

        instance.presentacion = validated_data.get('presentacion',
instance.presentacion)

        instance.stock = validated_data.get('stock', instance.stock)

        instance.save()

        return instance

```

ModelSerializer

Un ModelSerializer típicamente se refiere a un componente del framework Django REST (DRF). El framework Django REST es un popular conjunto de herramientas para construir APIs Web en aplicaciones Django. Proporciona un conjunto de herramientas y bibliotecas para simplificar el proceso de construcción de APIs, incluyendo serializadores.

La clase ModelSerializer proporciona un acceso directo que le permite crear automáticamente una clase Serializer con campos que corresponden a los campos del Modelo.

La clase ModelSerializer es la misma que una clase Serializer regular, excepto que:

- Se generará automáticamente un conjunto de campos para usted, basado en el modelo.
- Generará automáticamente validadores para el serializador, tales como `unique_together` validators.
- Incluye implementaciones simples por defecto de `.create()` y `.update()`.

Ahora vamos crear un serializador basado en un modelo, para ello escribimos lo siguiente:

Python

```
class ArtículoModelSerializer(serializers.ModelSerializer):  
  
    class Meta:  
  
        model = Artículo  
  
        fields = ['articulo_id', 'codigo_articulo', 'codigo_barras',  
                  'descripcion', 'presentacion', 'stock']  
  
        read_only_fields = ['articulo_id']
```

Nested-Serializers (Serializadores anidados)

Cuando se construyen APIs con Django REST Framework (DRF), el manejo de estructuras de datos complejas es un reto común. Las aplicaciones del mundo real a menudo implican múltiples modelos relacionados, y los clientes (como las aplicaciones frontales) normalmente esperan una respuesta única y unificada que incluya datos relacionados. Aquí es donde entran en juego los serializadores anidados.

Los serializadores anidados de DRF permiten representar objetos relacionados como parte de una única respuesta o crear/actualizar objetos relacionados en una única solicitud. Esto hace que su API sea más limpia, reduce el número de llamadas HTTP y simplifica el procesamiento de datos en el lado del cliente.

Vamos a crear serializadores anidados para la relación entre Artículo, GrupoArticulo y LineaArticulo, vamos a reemplazar el serializador ArtículoSerializer

Python

```
# api/serializers.py  
  
class GrupoArticuloSerializer(serializers.ModelSerializer):  
  
    class Meta:  
  
        model = GrupoArticulo
```

```

        fields = ['grupo_id', 'codigo_grupo', 'nombre_grupo']

class LineaArticuloSerializer(serializers.ModelSerializer):

    class Meta:

        model = LineaArticulo

        fields = ['linea_id', 'codigo_linea', 'nombre_linea']

class ListaPrecioSerializer(serializers.ModelSerializer):

    class Meta:

        model = ListaPrecio

        fields = ['precio_1', 'precio_2', 'precio_3', 'precio_4',
                  'precio_compra', 'precio_costo']

# api/serializers.py (continuación)

class ArticuloSerializer(serializers.ModelSerializer):

    grupo = GrupoArticuloSerializer(read_only=True)

    linea = LineaArticuloSerializer(read_only=True)

    # Campo para escribir los IDs

    grupo_id = serializers.UUIDField(write_only=True)

    linea_id = serializers.UUIDField(write_only=True)

    # Incluimos los precios

    precios = ListaPrecioSerializer(source='listaprecio', read_only=True)

    class Meta:

        model = Articulo

```

```
fields = [
    'articulo_id', 'codigo_articulo', 'codigo_barras',
    'descripcion', 'presentacion', 'stock',
    'grupo', 'linea', 'grupo_id', 'linea_id', 'precios'
]

def create(self, validated_data):
    grupo_id = validated_data.pop('grupo_id')
    linea_id = validated_data.pop('linea_id')

    # Obtener objetos reales
    grupo = GrupoArticulo.objects.get(pk=grupo_id)
    linea = LineaArticulo.objects.get(pk=linea_id)

    # Crear el artículo
    articulo = Articulo.objects.create(
        grupo=grupo,
        linea=linea,
        **validated_data
    )

    return articulo
```

Serializer simplificado para ListViews

Python

api/serializers.py (continuación)

```
class ArtículoListSerializer(serializers.ModelSerializer):

    grupo_nombre = serializers.CharField(source='grupo.nombre_grupo',
read_only=True)

    linea_nombre = serializers.CharField(source='linea.nombre_linea',
read_only=True)

    precio = serializers.DecimalField(
        source='listaprecio.precio_1',
        max_digits=10,
        decimal_places=2,
        read_only=True
    )

class Meta:

    model = Artículo

    fields = [
        'articulo_id', 'codigo_articulo', 'descripcion',
        'grupo_nombre', 'linea_nombre', 'stock', 'precio'
    ]
```

Serializer dinámico

Este serializer puede cambiar sus campos según el contexto (*api/serializers.py*).

Python

```
class DynamicFieldsModelSerializer(serializers.ModelSerializer):
    """
    Un ModelSerializer que toma un argumento adicional `fields` que
    controla qué campos deben incluirse.
    """
    def __init__(self, *args, **kwargs):
        # No olvides eliminar 'fields' de kwargs
        fields = kwargs.pop('fields', None)

        # Inicializa como lo harías normalmente
        super().__init__(*args, **kwargs)

        if fields is not None:
            # Elimina todos los campos que no están en la lista de campos
            allowed = set(fields)
            existing = set(self.fields)
            for field_name in existing - allowed:
                self.fields.pop(field_name)

class ArtículoDynamicSerializer(DynamicFieldsModelSerializer):
    grupo = serializers.SlugRelatedField(slug_field='nombre_grupo',
                                         read_only=True)

    linea = serializers.SlugRelatedField(slug_field='nombre_linea',
                                         read_only=True)
```

```
class Meta:
    model = Articulo
    fields = [
        'articulo_id', 'codigo_articulo', 'codigo_barras',
        'descripcion', 'presentacion', 'grupo', 'linea', 'stock'
    ]
```

Serializers para Órdenes

Python

```
class ItemOrdenSerializer(serializers.ModelSerializer):
    articulo_descripcion = serializers.CharField(source='articulo.descripcion',
        read_only=True)

    class Meta:
        model = ItemOrdenCompraCliente
        fields = [
            'item_id', 'nro_item', 'articulo', 'articulo_descripcion',
            'cantidad', 'precio_unitario', 'total_item'
        ]
```

```

class OrdenSerializer(serializers.ModelSerializer):

    items = ItemOrdenSerializer(source='item_orden_compra', many=True,
read_only=True)

    cliente_nombre = serializers.CharField(source='cliente.nombres',
read_only=True)

    estado_display = serializers.CharField(source='get_estado_display',
read_only=True)


class Meta:

    model = OrdenCompraCliente

    fields = [

        'pedido_id', 'nro_pedido', 'fecha_pedido', 'cliente',

        'cliente_nombre', 'vendedor', 'importe', 'estado',

        'estado_display', 'notas', 'items'

    ]

```

Serializer con validaciones personalizadas

Validar datos correctamente es clave para asegurar que una aplicación funcione de manera segura y confiable. En este caso, exploramos cómo implementar validaciones personalizadas en serializadores de Django REST Framework para garantizar que los datos cumplan con los requisitos específicos del negocio.

¿Cómo implementamos una validación personalizada en Django REST Framework?

Django ya ofrece validaciones básicas, como verificar que un campo sea un email. Sin embargo, para casos específicos, como asegurarse de que un correo pertenezca a un dominio corporativo, necesitamos crear validaciones personalizadas en el serializador. Esto lo logramos usando el

método `validate_<nombre_del_campo>`. Por ejemplo, para validar que un correo termine en `@example.com`, implementamos lo siguiente:

- Definimos el método `validate_email` dentro del serializador.
- Verificamos si el valor del campo contiene el dominio correcto.
- Si es válido, retornamos el valor; si no, levantamos una excepción con un mensaje de error.
- ¿Cómo manejar errores de validación en casos más complejos?
- Para validaciones que dependen de múltiples campos, como validar el número de contacto y el estado de vacaciones de un doctor, usamos el método general `validate`. Este método permite acceder a todos los campos del serializador en forma de diccionario y aplicar lógica personalizada.

Vamos a implementar algunas validaciones para nuestro modelo `Articulo`:

Python

```
class ArtículoCreateSerializer(serializers.ModelSerializer):

    grupo_id = serializers.UUIDField()

    linea_id = serializers.UUIDField()

    precio_1 = serializers.DecimalField(max_digits=12, decimal_places=2,
min_value=0)

class Meta:

    model = Articulo

    fields = [

        'codigo_articulo', 'codigo_barras', 'descripcion',

        'presentacion', 'grupo_id', 'linea_id', 'stock', 'precio_1'

    ]

    def validate_codigo_articulo(self, value):
```

```

    """
    Validar que el código de artículo sea único.
    """

    if Artículo.objects.filter(codigo_articulo=value).exists():
        raise serializers.ValidationError("Este código de artículo ya
existe.")

    return value

def validate(self, data):
    """
    Validar que la línea pertenezca al grupo.
    """

    grupo_id = data.get('grupo_id')
    linea_id = data.get('linea_id')

    try:
        linea = LineaArticulo.objects.get(pk=linea_id)

        if linea.grupo.grupo_id != grupo_id:
            raise serializers.ValidationError(
                {"linea_id": "La línea seleccionada no pertenece al
grupo."}
            )
    except LineaArticulo.DoesNotExist:
        raise serializers.ValidationError(
            {"linea_id": "La línea seleccionada no existe."}
        )

```

```
        return data

def create(self, validated_data):

    # Extraer precio_1
    precio_1 = validated_data.pop('precio_1')

    # Obtener objetos reales
    grupo_id = validated_data.pop('grupo_id')
    linea_id = validated_data.pop('linea_id')
    grupo = GrupoArticulo.objects.get(pk=grupo_id)
    linea = LineaArticulo.objects.get(pk=linea_id)

    # Generar un UUID para el artículo
    import uuid
    articulo_id = uuid.uuid4()

    # Crear el artículo
    articulo = Articulo.objects.create(
        articulo_id=articulo_id,
        grupo=grupo,
        linea=linea,
        **validated_data
    )
```

```
# Crear lista de precios

ListaPrecio.objects.create(

    articulo=articulo,

    precio_1=precio_1

)

return articulo
```

Vistas basadas en funciones

Al igual que con aplicaciones web, con DRF también podemos crear vistas basadas en funciones. En DRF (Django REST Framework), las vistas basadas en funciones son funciones de Python que manejan las solicitudes HTTP y devuelven respuestas HTTP. Son sencillas y fáciles de entender, especialmente para proyectos pequeños o cuando se necesita flexibilidad en el manejo de solicitudes. Vamos a crear algunas para nuestra API

Primero vamos a agregar nuestras referencias:

```
Python

from rest_framework.response import Response

from rest_framework import status
```

Luego vamos a implementar nuestras vistas, para ello escribimos lo siguiente:

```
Python

@api_view(['GET'])

def articulo_list(request):
```

```

"""
Lista todos los artículos.
"""

articulos = Artículo.objects.all()

serializer = ArtículoListSerializer(articulos, many=True)

return Response(serializer.data)


@api_view(['GET', 'PUT', 'DELETE'])
def articulo_detail(request, pk):
    """
    Obtener, actualizar o eliminar un artículo.
    """

    try:
        articulo = Artículo.objects.get(pk=pk)
    except Artículo.DoesNotExist:
        return Response(
            {"error": "Artículo no encontrado"},
            status=status.HTTP_404_NOT_FOUND
        )

    if request.method == 'GET':
        serializer = ArtículoSerializer(articulo)
        return Response(serializer.data)

```

```

elif request.method == 'PUT':

    serializer = ArtículoSerializer(articulo, data=request.data,
partial=True)

    if serializer.is_valid():

        serializer.save()

        return Response(serializer.data)

    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

elif request.method == 'DELETE':

    articulo.delete()

    return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['POST'])
def articulo_create(request):

    """
    Crear un nuevo artículo.
    """

    serializer = ArtículoCreateSerializer(data=request.data)

    if serializer.is_valid():

        articulo = serializer.save()

        return Response(

            ArtículoSerializer(articulo).data,

            status=status.HTTP_201_CREATED

        )

    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

Ahora debemos actualizar nuestra api/urls.py, agregando lo siguiente:

Python

```
from django.urls import path, include
from . import views

urlpatterns = [

    path('articulos/', views.articulo_list, name='articulo-list'),
    path('articulos/create/', views.articulo_create, name='articulo-create'),
    path('articulos/<uuid:pk>', views.articulo_detail,
name='articulo-detail'),
]
```

Vistas basadas en clases

En Django Rest Framework (DRF), las vistas basadas en clases (CBV) son un enfoque para definir vistas que se implementan como clases en lugar de funciones. Ofrecen una estructura más organizada para el manejo de métodos HTTP (GET, POST, PUT, DELETE, etc.) y la lógica relacionada, lo que facilita la creación de APIs RESTful robustas y mantenibles.

Ventajas de las vistas basadas en clases en DRF:

1. Organización:

Las CBV permiten agrupar la lógica para diferentes métodos HTTP dentro de una clase, haciendo el código más legible y fácil de mantener.

2. Reutilización:

DRF proporciona clases genéricas (e.g., ListAPIView, RetrieveAPIView) que pueden ser utilizadas como base para crear vistas personalizadas.

3. Herencia:

Puedes heredar de clases genéricas para añadir lógica específica a tus vistas, evitando la duplicación de código.

4. Mejor estructuración:

Las CBV facilitan la creación de APIs RESTful al separar la lógica de procesamiento de solicitudes y respuestas en diferentes métodos de la clase, como `get()`, `post()`, etc.

5. Acceso a contexto:

Las CBV tienen acceso a un contexto que incluye la solicitud, los argumentos y otros datos relevantes, lo que facilita la lógica de la vista.

Vamos a implementar las mismas funcionalidades usando vistas basadas en clases.

Python

```
class ArtículoListView(APIView):  
    """  
    Lista todos los artículos o crea uno nuevo.  
    """  
  
    def get(self, request, format=None):  
        articulos = Artículo.objects.all()  
        serializer = ArtículoListSerializer(articulos, many=True)  
        return Response(serializer.data)  
  
    def post(self, request, format=None):  
        serializer = ArtículoCreateSerializer(data=request.data)
```

```

        if serializer.is_valid():
            articulo = serializer.save()

            return Response(
                ArticuloSerializer(articulo).data,
                status=status.HTTP_201_CREATED
            )

        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

class ArticuloDetailView(APIView):
    """
    Obtener, actualizar o eliminar un artículo.
    """
    def get_object(self, pk):
        try:
            return Articulo.objects.get(pk=pk)
        except Articulo.DoesNotExist:
            raise Http404

    def get(self, request, pk, format=None):
        articulo = self.get_object(pk)
        serializer = ArticuloSerializer(articulo)
        return Response(serializer.data)

    def put(self, request, pk, format=None):

```

```

        articulo = self.get_object(pk)

        serializer = ArticuloSerializer(articulo, data=request.data,
partial=True)

        if serializer.is_valid():

            serializer.save()

            return Response(serializer.data)

        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

def delete(self, request, pk, format=None):

    articulo = self.get_object(pk)

    articulo.delete()

    return Response(status=status.HTTP_204_NO_CONTENT)

```

Ahora volvemos a actualizar nuestro archivo de urls `api/urls.py`

Python

```

# pos_project/urls.py

from django.urls import path, include

from . import views

urlpatterns = [

    path('articulos/', views.articulo_list, name='articulo-list'),

    path('articulos/create/', views.articulo_create, name='articulo-create'),

```

```

    path('articulos/<uuid:pk>/', views.articulo_detail,
         name='articulo-detail'),

    path('articulos/func/', views.articulo_list, name='articulo-list-func'),

    path('articulos/func/create/', views.articulo_create,
         name='articulo-create-func'),

    path('articulos/func/<uuid:pk>/', views.articulo_detail,
         name='articulo-detail-func'),

    # Vistas basadas en clases

    path('articulos/class/', views.ArticuloListView.as_view(),
         name='articulo-list-class'),

    path('articulos/class/<uuid:pk>/', views.ArticuloDetailView.as_view(),
         name='articulo-detail-class'),

]

```

Mixins y vistas genéricas

Los mixins son una forma de reutilizar la lógica de las vistas en el framework Django REST. Son clases que proporcionan una funcionalidad específica, como la autenticación o la paginación, que se pueden añadir fácilmente a cualquier vista incluyéndolas como clase base. Los Mixins se utilizan normalmente con las clases `APIView` o `GenericAPIView` para crear vistas con funcionalidades específicas.

En resumen, los Mixins se utilizan para añadir funcionalidad específica a las vistas y se pueden utilizar con `APIView` y `GenericAPIView` para crear vistas más complejas, mientras que `APIView` es la clase base para gestionar la solicitud y la respuesta de las vistas y se puede utilizar para vistas sencillas.

Para hacer uso de las mixins y vistas genéricas tenemos que agregar lo siguiente en nuestro archivo de `api/views.py`:

Python

```
from rest_framework import mixins  
from rest_framework import generics
```

Luego agregamos lo siguiente en nuestra vista:

Python

```
class ArtículoListCreateGeneric(mixins.ListModelMixin,  
                                mixins.CreateModelMixin,  
                                generics.GenericAPIView):  
  
    """  
    Lista todos los artículos o crea uno nuevo usando mixins.  
    """  
  
    queryset = Artículo.objects.all()  
  
    def get_serializer_class(self):  
        if self.request.method == 'POST':  
            return ArtículoCreateSerializer  
        return ArtículoListSerializer  
  
    def get(self, request, *args, **kwargs):  
        return self.list(request, *args, **kwargs)  
  
    def post(self, request, *args, **kwargs):  
        return self.create(request, *args, **kwargs)
```

```

def perform_create(self, serializer):

    # Personalizar cómo se guarda el objeto

    articulo = serializer.save()

    # Podríamos hacer más cosas aquí, como logging


class ArticuloDetailGeneric(mixins.RetrieveModelMixin,
                             mixins.UpdateModelMixin,
                             mixins.DestroyModelMixin,
                             generics.GenericAPIView):

    """
    Obtener, actualizar o eliminar un artículo usando mixins.
    """

    queryset = Articulo.objects.all()
    serializer_class = ArticuloSerializer

    def get(self, request, *args, **kwargs):

        return self.retrieve(request, *args, **kwargs)

    def put(self, request, *args, **kwargs):

        return self.update(request, *args, **kwargs)

    def delete(self, request, *args, **kwargs):

        return self.destroy(request, *args, **kwargs)

```

Vistas genéricas aún más simples

DRF proporciona vistas genéricas que combinan mixins comunes para simplificar el código.

Python

```
# Ahora usando las vistas genéricas más simplificadas

class ArtículoListCreateSimple(generics.ListCreateAPIView):
    """
    Lista todos los artículos o crea uno nuevo.
    Versión simplificada con vistas genéricas.
    """
    queryset = Artículo.objects.all()

    def get_serializer_class(self):
        if self.request.method == 'POST':
            return ArtículoCreateSerializer
        return ArtículoListSerializer

class ArtículoDetailSimple(generics.RetrieveUpdateDestroyAPIView):
    """
    Obtener, actualizar o eliminar un artículo.
    Versión simplificada con vistas genéricas.
    """
    queryset = Artículo.objects.all()
    serializer_class = ArtículoSerializer
```

Ahora, vamos a modificar nuestro archivo de `api/urls.py`:

Python

```
# api/urls.py (actualizado)

# Agregar estas rutas

urlpatterns += [

    # Vistas con mixins

    path('articulos/mixins/', views.ArticuloListCreateGeneric.as_view(),
         name='articulo-list-mixins'),

    path('articulos/mixins/<uuid:pk>/', views.ArticuloDetailGeneric.as_view(),
         name='articulo-detail-mixins'),

    # Vistas genéricas simplificadas

    path('articulos/generic/', views.ArticuloListCreateSimple.as_view(),
         name='articulo-list-generic'),

    path('articulos/generic/<uuid:pk>/', views.ArticuloDetailSimple.as_view(),
         name='articulo-detail-generic'),

]
```

ViewSet y Routers

Los ViewSets en Django REST Framework proporcionan una abstracción sobre las vistas normales, agrupando vistas relacionadas en una única clase. En lugar de definir funciones separadas para cada acción (como get, post, put, delete), los ViewSets utilizan acciones (como list, create, retrieve, update, destroy). Este enfoque reduce el código repetitivo y fomenta la coherencia en el diseño de la API, especialmente cuando se combina con enrutadores para la configuración automática de URL.

Existen varios tipos de ViewSets:

- **ViewSet:** La clase base, que requiere la definición explícita de todas las acciones.

- **ViewSet genérico:** Proporciona implementaciones por defecto para acciones comunes utilizando mixins, pero no incluye ninguna acción por defecto.
- **ReadOnlyModelViewSet:** Ofrece acciones de sólo lectura como listar y recuperar para un modelo.
- **ModelViewSet:** El tipo más completo, proporciona todas las operaciones CRUD para un modelo.

Los ViewSets funcionan perfectamente con los Routers, que generan automáticamente patrones de URL para todas las acciones definidas en el ViewSet. Esto elimina la necesidad de configurar manualmente las URL y garantiza una estructura de URL coherente en toda la API.

Ahora vamos a crear un par de viewsets en nuestra api, vamos al archivo de vistas (*api/views.py*)

Python

```
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
```

Luego vamos al final del archivo y escribimos lo siguiente:

Python

```
class ArtículoViewSet(viewsets.ModelViewSet):
    """
    Un viewset para ver y editar artículos.
    """
    queryset = Artículo.objects.all()

    def get_serializer_class(self):
        if self.action == 'create':
            return ArtículoCreateSerializer
```

```

        elif self.action == 'list':

            return ArtículoListSerializer

        return ArtículoSerializer

    @action(detail=True, methods=['get'])
    def precios(self, request, pk=None):
        """
        Endpoint personalizado para obtener precios de un artículo.
        GET /api/articulos/{id}/precios/
        """
        articulo = self.get_object()

        try:
            lista_precio = articulo.listaprecio

            serializer = ListaPrecioSerializer(lista_precio)

            return Response(serializer.data)

        except:
            return Response({"error": "No hay lista de precios"},
                status=status.HTTP_404_NOT_FOUND)

    @action(detail=False, methods=['get'])
    def bajo_stock(self, request):
        """
        Endpoint personalizado para obtener artículos con bajo stock.
        GET /api/articulos/bajo_stock/
        """
        articulos = Artículo.objects.filter(stock__lt=10)

```

```

        serializer = ArticuloListSerializer(articulos, many=True)

        return Response(serializer.data)

class OrdenViewSet(viewsets.ReadOnlyModelViewSet):
    """
    Un viewset para ver órdenes (solo lectura).
    """
    queryset = OrdenCompraCliente.objects.all()
    serializer_class = OrdenSerializer
    permission_classes = [IsAuthenticated]

    @action(detail=True, methods=['post'])
    def cancelar(self, request, pk=None):
        """
        Endpoint para cancelar una orden.
        POST /api/ordenes/{id}/cancelar/
        """
        from pos_project.choices import EstadoOrden

        orden = self.get_object()

        # Solo se pueden cancelar órdenes pendientes
        if orden.estado != EstadoOrden.PENDIENTE:
            return Response(

```

```
        {"error": "Solo se pueden cancelar órdenes pendientes."},
        status=status.HTTP_400_BAD_REQUEST
    )

    orden.estado = EstadoOrden.CANCELADA
    orden.save()

    serializer = self.get_serializer(orden)
    return Response(serializer.data)
```

Ahora vamos a modificar nuestro archivo de rutas, vamos a [api/urls.py](#) y modificamos lo siguiente:

Python

```
# pos_project/urls.py

from django.urls import path, include
from . import views
from rest_framework.routers import DefaultRouter

# Crear un router y registrar los viewsets
router = DefaultRouter()
router.register(r'articulos', views.ArticuloViewSet, basename='articulo')
router.register(r'ordenes', views.OrdenViewSet, basename='orden')
```

```
urlpatterns = [

    path('', include(router.urls)),

    path('articulos/', views.articulo_list, name='articulo-list'),

    path('articulos/create/', views.articulo_create, name='articulo-create'),

    path('articulos/<uuid:pk>', views.articulo_detail,
name='articulo-detail'),

    path('articulos/func/', views.articulo_list, name='articulo-list-func'),

    path('articulos/func/create/', views.articulo_create,
name='articulo-create-func'),

    path('articulos/func/<uuid:pk>', views.articulo_detail,
name='articulo-detail-func'),

    # Vistas basadas en clases

    path('articulos/class/', views.ArticuloListView.as_view(),
name='articulo-list-class'),

    path('articulos/class/<uuid:pk>', views.ArticuloDetailView.as_view(),
name='articulo-detail-class'),

    # Vistas con mixins

    path('articulos/mixins/', views.ArticuloListCreateGeneric.as_view(),
name='articulo-list-mixins'),

    path('articulos/mixins/<uuid:pk>', views.ArticuloDetailGeneric.as_view(),
name='articulo-detail-mixins'),

    # Vistas genéricas simplificadas
```

```
path('articulos/generic/', views.ArticuloListCreateSimple.as_view(),
name='articulo-list-generic'),

path('articulos/generic/<uuid:pk>', views.ArticuloDetailSimple.as_view(),
name='articulo-detail-generic'),

]
```

Filtrado

Django REST Framework (DRF) ofrece múltiples formas de filtrar conjuntos de consultas, lo que permite a los desarrolladores controlar qué objetos se devuelven en función de diversas condiciones. Ahora vamos a implementar filtrado en nuestra api, para ello debemos primero instalarlo

Python

```
pip install django-filter
```

A continuación, vamos a implementar sobre el viewset de articulos, para ello vamos al archivo

Python

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import filters
```

Luego buscamos la clase `ArticuloViewSet` y vamos a implementar filtros

Python

```
class ArticuloViewSet(viewsets.ModelViewSet):
```

```

"""
Un viewset para ver y editar artículos.
"""

queryset = Artículo.objects.all()

filter_backends = [DjangoFilterBackend, filters.SearchFilter,
filters.OrderingFilter]

filterset_fields = ['grupo', 'linea', 'stock']

search_fields = ['codigo_articulo', 'descripcion', 'codigo_barras']

ordering_fields = ['codigo_articulo', 'descripcion', 'stock']

```

Ahora vamos a configurar algunos parámetros en el archivo `pos_project/settings.py`. Agregamos lo siguiente:

Python

```

REST_FRAMEWORK = {

    'DEFAULT_PAGINATION_CLASS':
    'rest_framework.pagination.PageNumberPagination',

    'PAGE_SIZE': 10,

    'DEFAULT_AUTHENTICATION_CLASSES': [

        'rest_framework.authentication.BasicAuthentication',

        'rest_framework.authentication.SessionAuthentication',

    ],

    'DEFAULT_FILTER_BACKENDS': [

        'django_filters.rest_framework.DjangoFilterBackend',

    ],
}

```

```
}
```

Paginación personalizada

Vamos a crear una paginación personalizada. Para ello vamos a crear una clase que se encargue de gestionar todo el proceso de paginación agregamos el siguiente archivo `api/pagination.py`

Python

```
# api/pagination.py

from rest_framework.pagination import PageNumberPagination
from rest_framework.response import Response

class StandardResultsSetPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 100

class CustomPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 100

    def get_paginated_response(self, data):
        return Response({
            'links': {
```

```

        'next': self.get_next_link(),
        'previous': self.get_previous_link()
    },
    'count': self.page.paginator.count,
    'pages': self.page.paginator.num_pages,
    'current_page': self.page.number,
    'results': data
    })

```

Después, vamos a actualizar la vista para usar esta paginación, para ello vamos al archivo `api/views.py` y escogemos el método `ArticuloListCreateSimple`

Python

```

from .pagination import CustomPagination

class ArticuloListCreateSimple(generics.ListCreateAPIView):
    """
    Lista todos los artículos o crea uno nuevo.
    Versión simplificada con vistas genéricas.
    """
    queryset = Articulo.objects.all()
    pagination_class = CustomPagination

    def get_serializer_class(self):
        if self.request.method == 'POST':

```

```
        return ArtículoCreateSerializer
    return ArtículoListSerializer
```

Throttling (Limitación de tasa)

Throttling en Django, particularmente dentro del Django REST Framework (DRF), se utiliza para controlar la tasa de solicitudes que los clientes pueden hacer a una API. Es un mecanismo para proteger tu API de ser abrumada por peticiones excesivas, ya sean accidentales o maliciosas. El throttling es similar a los permisos, ya que determina si una solicitud debe ser autorizada, pero se centra en los estados temporales y las tasas de solicitud.

Primero vamos a configurar su uso en el archivo de pos_project/[settings.py](#)

Python

```
# pos_project/settings.py (actualizar)

REST_FRAMEWORK = {
    # ... configuración existente ...
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle'
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '10/minute',
        'user': '100/minute',
        'burst': '5/minute',
        'sustained': '50/day'
    }
}
```

```
}
```

Luego tenemos que crear la clase personalizada de throttling, para ello agregamos el archivo `api/throttling.py` y escribimos lo siguiente:

Python

```
# api/throttling.py

from rest_framework.throttling import UserRateThrottle, AnonRateThrottle

class BurstRateThrottle(AnonRateThrottle):
    scope = 'burst'

class SustainedRateThrottle(UserRateThrottle):
    scope = 'sustained'
```

Ahora debemos de actualizar la vista para poder usar la clase creada, para este ejemplo usaremos las funciones `articulo_list` y `ArticuloViewSet`

Python

```
from .throttling import BurstRateThrottle, SustainedRateThrottle

@api_view(['GET'])

def articulo_list(request):
    """
    Lista todos los artículos.
    """
```

```

"""

throttle_classes = [BurstRateThrottle]

for throttle_class in throttle_classes:
    throttle = throttle_class()

    # El error está aquí, no debemos pasar 'self' sino 'view=None'
    if not throttle.allow_request(request, view=None):
        return Response(
            {"error": "Demasiadas solicitudes, intente más tarde."},
            status=status.HTTP_429_TOO_MANY_REQUESTS
        )

articulos = Artículo.objects.all()
serializer = ArtículoListSerializer(articulos, many=True)
return Response(serializer.data)

class ArtículoViewSet(viewsets.ModelViewSet):
    """
    Un viewset para ver y editar artículos.
    """

    queryset = Artículo.objects.all()

    filter_backends = [DjangoFilterBackend, filters.SearchFilter,
filters.OrderingFilter]

    filterset_fields = ['grupo', 'linea', 'stock']

```

```
search_fields = ['codigo_articulo', 'descripcion', 'codigo_barras']

ordering_fields = ['codigo_articulo', 'descripcion', 'stock']

throttle_classes = [SustainedRateThrottle]
```

Seguridad y Autenticación

DRF proporciona varios esquemas de autenticación. Vamos a implementar algunos.

Autenticación por Token

Primero, instalemos el paquete necesario:

Python

```
pip install djangorestframework-simplejwt
```

Ahora vamos a actualizar [settings.py](#)

Python

```
# pos_project/settings.py
```

```
INSTALLED_APPS = [

    # ... apps existentes

    'rest_framework',

    'rest_framework_simplejwt',

    'api',
```

```

]

REST_FRAMEWORK = {

    # ... configuración existente

    'DEFAULT_AUTHENTICATION_CLASSES': [

        'rest_framework_simplejwt.authentication.JWTAuthentication',

        'rest_framework.authentication.SessionAuthentication',

        'rest_framework.authentication.BasicAuthentication',

    ],

}

# Configuración JWT

from datetime import timedelta

SIMPLE_JWT = {

    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),

    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),

    'ROTATE_REFRESH_TOKENS': False,

    'BLACKLIST_AFTER_ROTATION': True,

    'ALGORITHM': 'HS256',

    'SIGNING_KEY': SECRET_KEY,

    'VERIFYING_KEY': None,

    'AUTH_HEADER_TYPES': ('Bearer',),

    'USER_ID_FIELD': 'username',

    'USER_ID_CLAIM': 'user_id',

```

```
'AUTH_TOKEN_CLASSES': ('rest_framework_simplejwt.tokens.AccessToken',),  
'TOKEN_TYPE_CLAIM': 'token_type',  
}
```

Ahora agreguemos endpoints en `api/urls.py` para obtener tokens:

```
Python  
  
from rest_framework_simplejwt.views import (  
    TokenObtainPairView,  
    TokenRefreshView,  
    TokenVerifyView,  
)
```

Luego, hacia el final del archivo agregamos lo siguiente:

```
Python  
  
urlpatterns += [  
    path('token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),  
    path('token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),  
    path('token/verify/', TokenVerifyView.as_view(), name='token_verify'),  
]
```

Permisos en Django

En Django Rest Framework, existen varios métodos para abordar el tema de permisos. A nivel básico, añades el atributo `permission_classes` a las clases que hereden de `APIView` para incluir los permisos de acceso. Los tipos de permisos que recibe este atributo están en `rest_framework.permissions`

Hay otra forma más avanzada de abordar los permisos y es creando clases de permisos que hereden de la clase base de Rest Framework (`rest_framework.permissions.BasePermission`). Precisamente esta última es la que vamos a ver, para ello vamos a crear el archivo `api/permissions.py`

Python

```
# api/permissions.py

from rest_framework import permissions

class IsAdminOrReadOnly(permissions.BasePermission):
    """
    Permiso que permite solo a administradores editar.
    Los usuarios no autenticados o no administradores solo pueden ver.
    """
    def has_permission(self, request, view):
        # Permitir peticiones GET, HEAD, OPTIONS a cualquiera
        if request.method in permissions.SAFE_METHODS:
            return True

        # Solo permitir escritura si el usuario es staff/admin
        return request.user and request.user.is_staff
```

```

class IsOwnerOrReadOnly(permissions.BasePermission):
    """
    Permiso que permite solo a propietarios de un objeto editarlo.
    """
    def has_object_permission(self, request, view, obj):
        # Permitir peticiones de solo lectura a cualquiera
        if request.method in permissions.SAFE_METHODS:
            return True

        # Solo permitir edición al propietario del objeto
        # Ajustar según el modelo (aquí suponemos que tiene campo creado_por)
        return obj.creado_por == request.user

```

A continuación, vamos a ver cómo usamos o aplicamos estos permisos para ello vamos a nuestro archivo `api/views.py` y agregamos las referencias

Python

```

from rest_framework.permissions import IsAuthenticated, IsAdminUser
from .permissions import IsAdminOrReadOnly, IsOwnerOrReadOnly

```

Luego buscamos las clases `ArticuloViewSet` y `OrdenViewSet` para aplicar los permisos escribiendo lo siguiente:

Python

```

class ArticuloViewSet(viewsets.ModelViewSet):

```

```

"""
Un viewset para ver y editar artículos.
"""

queryset = Artículo.objects.all()

filter_backends = [DjangoFilterBackend, filters.SearchFilter,
filters.OrderingFilter]

filterset_fields = ['grupo', 'linea', 'stock']

search_fields = ['codigo_articulo', 'descripcion', 'codigo_barras']

ordering_fields = ['codigo_articulo', 'descripcion', 'stock']

throttle_classes = [SustainedRateThrottle]

permission_classes = [IsAdminOrReadOnly] # Solo admin puede editar


class OrdenViewSet(viewsets.ReadOnlyModelViewSet):
    """
    Un viewset para ver órdenes (solo lectura).
    """

    queryset = OrdenCompraCliente.objects.all()

    serializer_class = OrdenSerializer

    permission_classes = [IsAuthenticated] # Solo usuarios autenticados


    def get_queryset(self):
        """
        Este viewset debe devolver solo las órdenes del usuario actual,
        a menos que sea staff.
        """

```

```

user = self.request.user

if user.is_staff:

    return OrdenCompraCliente.objects.all()

# Para usuarios normales, mostrar solo sus propias órdenes

# Suponiendo que el cliente está relacionado con el email del usuario

return
OrdenCompraCliente.objects.filter(cliente__correo_electronico=user.email)

```

Validaciones adicionales

Ya vimos algunas validaciones básicas en nuestro serializer `ArticuloCreateSerializer`. Vamos a ampliarlas en archivo `api/serializers.py`:

Python

```

# api/serializers.py (actualizar)

class ArticuloCreateSerializer(serializers.ModelSerializer):

    grupo_id = serializers.UUIDField()

    linea_id = serializers.UUIDField()

    precio_1 = serializers.DecimalField(max_digits=12, decimal_places=2,
min_value=0)

class Meta:

    model = Articulo

    fields = [

        'codigo_articulo', 'codigo_barras', 'descripcion',

        'presentacion', 'grupo_id', 'linea_id', 'stock', 'precio_1'

```

```

]

def validate_codigo_articulo(self, value):
    """
    Validar que el código de artículo sea único y cumpla con el formato.
    """
    if Artículo.objects.filter(codigo_articulo=value).exists():
        raise serializers.ValidationError("Este código de artículo ya
existe.")

    # Validar formato del código (por ejemplo, debe tener al menos 4
caracteres)
    if len(value) < 4:
        raise serializers.ValidationError("El código debe tener al menos 4
caracteres.")

    return value

def validate_descripcion(self, value):
    """
    Validar la descripción del artículo.
    """
    # Verificar que la descripción no sea demasiado corta
    if len(value) < 5:
        raise serializers.ValidationError("La descripción es demasiado
corta.")

```

```

        # Verificar que la descripción no sea demasiado larga
        if len(value) > 150:
            raise serializers.ValidationError("La descripción no debe exceder
los 150 caracteres.")

        return value

def validate_stock(self, value):
    """
    Validar que el stock no sea negativo.
    """
    if value < 0:
        raise serializers.ValidationError("El stock no puede ser
negativo.")

    return value

def validate(self, data):
    """
    Validaciones a nivel de objeto completo.
    """
    grupo_id = data.get('grupo_id')
    linea_id = data.get('linea_id')

    # Validar que el grupo existe
    try:

```

```

        grupo = GrupoArticulo.objects.get(pk=grupo_id)
except GrupoArticulo.DoesNotExist:
    raise serializers.ValidationError(
        {"grupo_id": "El grupo seleccionado no existe."}
    )

# Validar que la línea existe y pertenece al grupo
try:
    linea = LineaArticulo.objects.get(pk=linea_id)
    if linea.grupo.grupo_id != grupo_id:
        raise serializers.ValidationError(
            {"linea_id": "La línea seleccionada no pertenece al
grupo."}
        )
except LineaArticulo.DoesNotExist:
    raise serializers.ValidationError(
        {"linea_id": "La línea seleccionada no existe."}
    )

# Validar que el precio es coherente con el costo
precio_1 = data.get('precio_1')

return data

```

Versionamiento de APIs

El versionamiento de APIs REST permite a los desarrolladores realizar cambios y mejoras en sus APIs sin interrumpir a los clientes existentes. Al etiquetar diferentes versiones de la API, los desarrolladores pueden proporcionar a los clientes la opción de apuntar a una versión específica, lo que garantiza que sus aplicaciones seguirán funcionando correctamente incluso después de los cambios realizados en esta.

Vamos a actualizar el archivo [settings.py](#)

Python

```
# pos_project/settings.py (actualizar)

REST_FRAMEWORK = {
    # ... configuración existente

    'DEFAULT_VERSIONING_CLASS': 'rest_framework.versioning.URLPathVersioning',
    'DEFAULT_VERSION': 'v1',
    'ALLOWED_VERSIONS': ['v1', 'v2'],
    'VERSION_PARAM': 'version',
}
```

Ahora vamos a crear un nuevo archivo de vistas que usaremos para la nueva versión de nuestra api `api/views_v2.py`

Python

```
# api/views_v2.py

from rest_framework import viewsets, filters, status
from rest_framework.response import Response
from rest_framework.decorators import action
```

```

from django_filters.rest_framework import DjangoFilterBackend

from core.models import Artículo,
OrdenCompraCliente, GrupoArticulo, LineaArticulo

from .serializers import ArtículoSerializer, ArtículoListSerializer,
OrdenSerializer

from .permissions import IsAdminOrReadOnly

from .throttling import SustainedRateThrottle

from .pagination import CustomPagination


class ArtículoViewSetV2(viewsets.ModelViewSet):
    """
    API v2: Un viewset para ver y editar artículos.

    Cambios en V2:
    - Incluye información extendida de cada artículo
    - Paginación personalizada
    - Ordenamiento por defecto por código
    """

    queryset = Artículo.objects.all().order_by('codigo_articulo')

    filter_backends = [DjangoFilterBackend, filters.SearchFilter,
filters.OrderingFilter]

    filterset_fields = ['grupo', 'linea', 'stock']

    search_fields = ['codigo_articulo', 'descripcion', 'codigo_barras']

    ordering_fields = ['codigo_articulo', 'descripcion', 'stock']

    throttle_classes = [SustainedRateThrottle]

    permission_classes = [IsAdminOrReadOnly]

    pagination_class = CustomPagination

```

```

def get_serializer_class(self):
    if self.action == 'list':
        return ArtículoListSerializer
    return ArtículoSerializer

@action(detail=True, methods=['get'])
def precios(self, request, pk=None):
    """
    Endpoint personalizado para obtener precios de un artículo.
    GET /api/v2/articulos/{id}/precios/
    """
    articulo = self.get_object()
    try:
        lista_precio = articulo.listaprecio
        serializer = ListaPrecioSerializer(lista_precio)
        return Response(serializer.data)
    except:
        return Response({"error": "No hay lista de precios"},
            status=status.HTTP_404_NOT_FOUND)

@action(detail=False, methods=['get'])
def stats(self, request):
    """
    Endpoint nuevo en V2 que proporciona estadísticas sobre artículos.
    GET /api/v2/articulos/stats/
    """

```

```

    """

    total_articulos = Artículo.objects.count()

    bajo_stock = Artículo.objects.filter(stock__lt=10).count()

    # Artículos por grupo

    from django.db.models import Count

    grupos = GrupoArticulo.objects.annotate(

        articulos_count=Count('articulo')

    ).values('nombre_grupo', 'articulos_count')

    return Response({

        'total_articulos': total_articulos,

        'bajo_stock': bajo_stock,

        'distribucion_por_grupo': grupos

    })

class OrdenViewSetV2(viewsets.ReadOnlyModelViewSet):

    """

    API v2: Un viewset para ver órdenes (solo lectura).

    """

    queryset = OrdenCompraCliente.objects.all()

    serializer_class = OrdenSerializer

    pagination_class = CustomPagination

```

```

def get_queryset(self):
    """
    Este viewset debe devolver solo las órdenes del usuario actual,
    a menos que sea staff.
    """
    user = self.request.user
    if user.is_staff:
        return OrdenCompraCliente.objects.all()

    # Para usuarios normales, mostrar solo sus propias órdenes
    return
OrdenCompraCliente.objects.filter(cliente__correo_electronico=user.email)

@action(detail=True, methods=['post'])
def cancelar(self, request, pk=None):
    """
    Endpoint para cancelar una orden.
    POST /api/v2/ordenes/{id}/cancelar/
    """
    from pos_project.choices import EstadoOrden

    orden = self.get_object()

    # Solo se pueden cancelar órdenes pendientes
    if orden.estado != EstadoOrden.PENDIENTE:
        return Response(

```

```

        {"error": "Solo se pueden cancelar órdenes pendientes."},
        status=status.HTTP_400_BAD_REQUEST
    )

orden.estado = EstadoOrden.CANCELADA
orden.save()

serializer = self.get_serializer(orden)
return Response(serializer.data)

@api_action(detail=False, methods=['get'])
def pendientes(self, request):
    """
    Endpoint nuevo en V2 que muestra solo órdenes pendientes.
    GET /api/v2/ordenes/pendientes/
    """
    from pos_project.choices import EstadoOrden

    ordenes = self.get_queryset().filter(estado=EstadoOrden.PENDIENTE)
    page = self.paginate_queryset(ordenes)

    if page is not None:
        serializer = self.get_serializer(page, many=True)
        return self.get_paginated_response(serializer.data)

    serializer = self.get_serializer(ordenes, many=True)
    return Response(serializer.data)

```

Ahora vamos a modificar nuestro de archivo de rutas

Python

```
from . import views_v2

app_name = 'api'

# Crear routers para cada versión

router_v1 = DefaultRouter()

router_v1.register(r'articulos', views.ArticuloViewSet, basename='articulo')

router_v1.register(r'ordenes', views.OrdenViewSet, basename='orden')

router_v2 = DefaultRouter()

router_v2.register(r'articulos', views_v2.ArticuloViewSetV2,
basename='articulo')

router_v2.register(r'ordenes', views_v2.OrdenViewSetV2, basename='orden')
```

—